

Critical Audit of SZ-DBC: Detailed Balance Checker for Lattice MCMC

A referee-style review of correctness, completeness, and rigor

Independent Technical Review

June 2026

Contents

1	Overview and Scope	3
2	The BFS Engine and Transition Weight Calculation	3
2.1	Background: How Weights Are Assigned	3
2.2	The Bug	3
2.3	When Does This Affect the DB Check?	4
2.4	The Fix	5
3	The D4 Rotational Invariance Check	5
3.1	Current Implementation	5
3.2	False-Positive Risk	5
3.3	Why the Translational τ -Trick Cannot Be Directly Adapted for D4	6
3.4	A Rigorous Algebraic D4 Check via EBE Extension	7
4	The τ-BFS: Actual Failure Modes	7
4.1	What the τ -BFS Detects	7
4.2	Internal Mathematica Calls to <code>RandomInteger</code> Cause Spurious Errors . . .	7
4.3	The τ -Check from Orbit Representatives Is Sound	8
4.4	The τ -Check Verifies Weights But Not Next-State Consistency	8
5	The Ergodicity Check	9
6	The EBE Exact Check	9
6.1	Correctness of the Arrangement BFS	9
6.2	Strict Inequalities and Boundary Omission	9
6.3	Correctness of <code>\$dbcIsExpZero</code>	10
7	Minor Issues	10
7.1	Time-Limited BFS Returns Partial Leaves Without Error	10
7.2	Coupling Symmetry Rule Persists Across Runs in a Live Kernel	11
7.3	The <code>seqBernoulli</code> Weight Calculation Is Correct But Fragile	11
7.4	The <code>RandomVariate[NormalDistribution]</code> Approximation	11

8	Documentation Inconsistencies	11
8.1	Help Text vs. Code: <code>ebeMaxK</code> Default	11
8.2	Description of τ -Failure for <code>quadratic_field.wl</code>	11
8.3	Claim About False-Positive Probability	12
9	Fixes Applied and Remaining Issues	12
9.1	Fix 1: RandomChoice weight correction (critical)	12
9.2	Fix 2: Ergodicity BFS starts from correct seed state	13
9.3	Fix 3: <code>ebeMaxK</code> documentation error	13
9.4	All four open issues now resolved	13
9.5	Summary of status after second revision	14
10	Summary Table	15
11	Conclusions	15

1 Overview and Scope

SZ-DBC is a software tool that automatically verifies detailed balance for lattice MCMC algorithms. The checker intercepts random-number calls during a symbolic BFS, builds a symbolic transition matrix, and then checks the detailed balance condition

$$T(s \rightarrow t) \pi(s) = T(t \rightarrow s) \pi(t), \quad \pi(s) \propto e^{-\beta E(s)}, \quad (1)$$

exactly (EBE mode) or probabilistically (SZ fallback) over all coupling-constant parameter regions.

This report presents a systematic audit of every step of the pipeline. All claims were verified by reading the source code (`dbc_core.wl`, `check.wls`) and by running targeted experiments using the Mathematica kernel. The findings are ordered roughly by severity.

Bottom line. The checker is broadly sound but contains one *correctness-critical* bug (Section 2) and one *design weakness* that can produce false results in ways not disclosed to the user (Section 4). The D4 check is numerical and probabilistic; a rigorous algebraic alternative exists and is described in Section 3.3. Several minor issues are documented in Sections 7 and 8.

2 The BFS Engine and Transition Weight Calculation

2.1 Background: How Weights Are Assigned

The BFS engine (`RunWithBitsAT`) replaces all random-number calls with symbolic tokens. For `RandomReal[]`, it uses an interval-tracking scheme: each call creates a fresh token U_j representing a uniform on $[0, 1]$. When the code compares $U_j < p$, a new bit b is read from the bit-string. If $b = 1$ the interval narrows to $[0, p]$ and the path weight is multiplied by $p/(1 - 0) = p$; if $b = 0$ the interval becomes $[p, 1]$ and weight is multiplied by $1 - p$. After the algorithm terminates, the leaf weight equals the probability of having followed that exact bit sequence given the coupling constants and β .

For `RandomChoice[list]` with $n = \text{Length}[\text{list}]$, the code reads $k = \lceil \log_2 n \rceil$ bits and computes $\text{val} \in \{0, \dots, 2^k - 1\}$. If $\text{val} \geq n$ the path is discarded as `$dbc$outOfRange`; otherwise the element `list[[val+1]]` is chosen. Each bit read multiplies the current weight by $\frac{1}{2}$. For an n -element list, the resulting weight per valid leaf is $(\frac{1}{2})^k$.

2.2 The Bug

Issue 1 (Incorrect weights for non-power-of-2 selection pools). When n is not a power of 2, the BFS assigns weight $\frac{1}{2^k}$ to each outcome, whereas the actual probability is $\frac{1}{n}$. This is a systematic relative error of

$$\frac{1/2^k}{1/n} = \frac{n}{2^k} \neq 1 \quad \text{when } n \text{ is not a power of 2.}$$

This was verified empirically: `RandomChoice[{a,b,c}]` ($n = 3$, $k = 2$) produces three leaves each with weight $\frac{1}{4}$, summing to $\frac{3}{4}$, not 1.

The following output was obtained from a direct call to `RunWithBitsAT`:

```

bits={0,0}    result={10, 1/4}    (* should be 1/3 *)
bits={0,1}    result={20, 1/4}
bits={1,0}    result={30, 1/4}
bits={1,1}    result=$dbc$outOfRange
Weight sum: 3/4    (* should be 1 *)

```

2.3 When Does This Affect the DB Check?

The detailed balance check evaluates $T(s \rightarrow t)e^{-\beta E_s} - T(t \rightarrow s)e^{-\beta E_t} = 0$. If both $T(s \rightarrow t)$ and $T(t \rightarrow s)$ incur the *same* multiplicative error $\alpha = n/2^k$, the condition becomes $\alpha T_{\text{true}}(s \rightarrow t)e^{-\beta E_s} = \alpha T_{\text{true}}(t \rightarrow s)e^{-\beta E_t}$, which is equivalent to the correct condition. Hence the bug is harmless whenever the selection-pool size is the same in both directions.

However, if the forward and reverse transitions use selection pools of *different* non-power-of-2 sizes n_f and n_r , the BFS assigns:

$$T_{\text{BFS}}(s \rightarrow t) = T_{\text{true}}(s \rightarrow t) \cdot \frac{n_f}{2^{k_f}}, \quad T_{\text{BFS}}(t \rightarrow s) = T_{\text{true}}(t \rightarrow s) \cdot \frac{n_r}{2^{k_r}}.$$

The checker evaluates the *BFS* detailed balance condition:

$$T_{\text{true}}(s \rightarrow t) \cdot \frac{n_f}{2^{k_f}} e^{-\beta E_s} = T_{\text{true}}(t \rightarrow s) \cdot \frac{n_r}{2^{k_r}} e^{-\beta E_t}.$$

This differs from the *true* detailed balance condition by the factor $\frac{n_f/2^{k_f}}{n_r/2^{k_r}}$. When this ratio is not 1, the checker operates on a shifted DB condition and can produce both false positives and false negatives.

Concrete false-pass example. Consider two states s_1 and s_2 with *equal* energies $E(s_1) = E(s_2) = 0$. An algorithm proposes $s_1 \rightarrow s_2$ by selecting uniformly from a 3-element list ($n_f = 3$, $k_f = 2$, $\alpha_f = 3/4$), and proposes $s_2 \rightarrow s_1$ from a 4-element list ($n_r = 4$, $k_r = 2$, $\alpha_r = 1$, a power of 2, no error). True probabilities: $T_{\text{true}}(s_1 \rightarrow s_2) = 1/3$, $T_{\text{true}}(s_2 \rightarrow s_1) = 1/4$.

With equal energies, true DB requires $T(s_1 \rightarrow s_2) = T(s_2 \rightarrow s_1)$, i.e. $1/3 = 1/4$, which is *false*. The algorithm violates detailed balance.

The BFS assigns $T_{\text{BFS}}(s_1 \rightarrow s_2) = 1/4$ (three leaves at weight $1/4$ each) and $T_{\text{BFS}}(s_2 \rightarrow s_1) = 1/4$ (four leaves at weight $1/4$ each). The BFS DB condition becomes $1/4 \cdot e^0 = 1/4 \cdot e^0$, i.e. $1/4 = 1/4$, which is *true*. The checker reports **PASS for a broken algorithm**.

More generally, if $n_f = 3$ (forward) and $n_r = 5$ (reverse):

- BFS factor ratio: $(n_f/2^{k_f})/(n_r/2^{k_r}) = (3/4)/(5/8) = 6/5 \neq 1$.
- The checker checks the wrong DB condition, shifted by factor $6/5$.

Impact on the provided examples. In all provided examples (`single_metropolis.wl`, `kawasaki.wl`, `vmmc_2d.wl`), the selection pool sizes are:

- Particle selection: $n = 3$ (always 3 particles) $\Rightarrow k = 2$, $\alpha = 3/4$ in both directions.
- Direction selection: $n = 8$ (power of 2) $\Rightarrow$ no error.

The factor $\alpha = 3/4$ cancels between forward and reverse, so the DB results for the provided examples are correct despite the bug. However, the bug is a latent correctness defect for algorithms where selection-pool size varies between forward and reverse transitions—precisely the class of Metropolis-Hastings algorithms with asymmetric proposals.

2.4 The Fix

The correct weight for a valid outcome of `RandomChoice[list]` with n elements is $1/n$. The BFS should renormalize the weight after reading k bits:

```
(* After choosing outcome: multiply weight by 2^k / n to
   renormalize *)
weight *= (2^k / n);
```

This converts the $1/2^k$ per-leaf weight to $1/n$, giving correct absolute probabilities and making the DB check valid for all algorithms regardless of whether n is a power of 2.

An equivalent fix: for `RandomChoice[list]`, instead of the rejection-sampling approach (read $k = \lceil \log_2 n \rceil$ bits, discard if out of range), use the interval-tracking approach analogous to `RandomReal[]`. Partition $[0, 1]$ into n equal intervals; use a single real token U with sequentially refined bits to identify which interval U falls in. The resulting leaf weight is exactly $1/n$. This is in fact what the `seqBernoulli` function does for weighted `RandomChoice`, but the unweighted case skips this and goes directly to bit-reading.

3 The D4 Rotational Invariance Check

3.1 Current Implementation

The D4 check (Section 7d of `dbc_core.wl`) verifies that $T(s \rightarrow t) = T(R \cdot s \rightarrow R \cdot t)$ for all D4 elements R and communicating pairs (s, t) , by evaluating both sides numerically at $\text{nReps} \in [5, 10]$ random coupling-constant points and checking $|T(s \rightarrow t) - T(R \cdot s \rightarrow R \cdot t)| < 10^{-6}$ (relative).

This is a probabilistic check. As acknowledged in the README, it can in principle produce false positives (declaring D4 when it does not hold) or false negatives (declaring no D4 when it does hold), though both are claimed to be negligibly unlikely in practice.

3.2 False-Positive Risk

The risk of a false positive deserves careful analysis. The entries $T(s \rightarrow t)$ and $T(R \cdot s \rightarrow R \cdot t)$ are rational functions of the coupling constants (after EBE region fixation). Their difference $\Delta(J) = T(s \rightarrow t) - T(R \cdot s \rightarrow R \cdot t)$ is a nonzero rational function for a genuinely asymmetric algorithm. The zero set $\{\Delta = 0\}$ has measure zero in coupling-constant space, so random points will lie outside it with probability 1 (for exact arithmetic).

With floating-point tolerance 10^{-6} , a false positive occurs if $|\Delta(J_i)| < 10^{-6} \cdot \max(|T(s \rightarrow t)|, |T(R \cdot s \rightarrow R \cdot t)|, 10^{-30})$ at all nReps test points. This requires either:

1. The violation Δ to be uniformly very small (nearly symmetric algorithm), or
2. All random coupling points to lie near the zero surface by coincidence.

For algorithms that are “accidentally close” to D4 (e.g., D4 violated only via a small perturbation term), the violation could be smaller than the tolerance 10^{-6} at all 10 test points, producing a false positive. Such an algorithm would then have its orbits incorrectly reduced from 56 to 8 representatives, causing the subsequent DB check to operate on an incorrect (smaller) transition graph.

Consequence of a false positive in D4. If the D4 check incorrectly declares symmetry, the orbit reduction from 56 to 8 representatives is unwarranted. The DB

check then runs on 8-rep BFS output and constructs the full T matrix via the false D4 symmetry assumption. States that are not in the same D4-orbit but happen to be related by the incorrectly applied group action will have their transition probabilities conflated. The DB check may report PASS when detailed balance is in fact violated in the non-symmetric part of the parameter space.

3.3 Why the Translational τ -Trick Cannot Be Directly Adapted for D4

The translational invariance check works by augmenting all particle positions with a symbolic offset $\tau = (\tau_r, \tau_c)$:

$$p_i \mapsto p_i + \tau.$$

In a translation-invariant algorithm, all spatial operations use *pairwise differences*:

$$(p_i + \tau) - (p_j + \tau) = p_i - p_j,$$

so τ cancels algebraically from every transition probability. Running the BFS once from a τ -augmented state, and checking that no leaf weight contains τ , certifies translation invariance for the entire state space.

For D4, an analogous augmentation would require a symbolic rotation R such that:

$$p_i \mapsto R p_i,$$

and the algorithm's outputs should satisfy $T(R \cdot s \rightarrow R \cdot t) = T(s \rightarrow t)$. The fundamental obstacle to the τ -style algebraic trick is that D4 is a *discrete, non-additive* group:

1. **Discreteness.** The D4 group has exactly 8 elements. There is no continuous parameter interpolating between them. The translation group, in contrast, acts via a continuous offset $\tau \in \mathbb{R}^2$, which can be introduced as a symbolic variable.
2. **Non-additivity.** Translation acts *additively*: $p \mapsto p + \tau$. This means τ enters positions linearly and is canceled by subtraction. D4 rotations act *multiplicatively* via 2×2 matrices: $p \mapsto R p$. Even if one introduces a symbolic rotation matrix $\mathbf{R}(\theta)$ parameterized by an angle θ , the constraint that θ takes only the values $\{0, 90, 180, 270\}$ degrees means the cancellation is not algebraic over the reals—it would require verifying $T(\mathbf{R} s \rightarrow \mathbf{R} t) = T(s \rightarrow t)$ for each of 8 specific choices of $\mathbf{R}$, which is no simpler than the original problem.
3. **Distance invariance.** For isotropic interactions, the pairwise energy satisfies $E(R \cdot s) = E(s)$ because $|R p_i - R p_j|^2 = |p_i - p_j|^2$. One might hope to exploit this algebraically. However, the *transition probabilities* also depend on the order in which particles are processed (e.g., which particle is selected first by `RandomChoice[state]`), and this ordering is not preserved by D4 rotations unless the algorithm is carefully written to use a canonical ordering. Verifying algebraically that the ordering is canonical under all D4 actions is itself a non-trivial problem.

3.4 A Rigorous Algebraic D4 Check via EBE Extension

Although the τ -style trick cannot be directly applied, a *fully rigorous algebraic check* is achievable by extending the existing EBE machinery. The key observation is that after EBE Phase 2 (arrangement BFS enumerating all parameter regions), the transition weights $T(s \rightarrow t)$ are known as exact rational numbers within each region. The D4 symmetry condition $T(s \rightarrow t) = T(R \cdot s \rightarrow R \cdot t)$ then reduces to a finite set of exact rational comparisons.

Proposed algorithm:

1. Run the translation-only BFS (already done in Step 5 of the current pipeline).
2. Run EBE Phase 1 and Phase 2 (already done in the DB check) to enumerate all T feasible parameter regions with rational representatives J^* .
3. For each of the T regions, substitute J^* into all leaf weights (already done in EBE Phase 3). This yields exact rational values for every $T_{\text{BFS}}(s \rightarrow t)$.
4. For each communicating pair (s, t) and each of the 7 non-identity D4 elements R : check whether $T(s \rightarrow t) = T(R \cdot s \rightarrow R \cdot t)$ as an exact rational equality.
5. If all equalities hold across all T regions and all 7 D4 elements, D4 symmetry is algebraically certified.

This approach has cost $O(T \times 7 \times |\text{pairs}|)$ rational comparisons, which is comparable to (and no more expensive than) the DB check itself. It would replace the 10-point numerical check with an exact certificate.

A further simplification is possible: the DB check for correctness already enumerates all regions. The D4 check can piggyback on this enumeration at negligible marginal cost.

4 The τ -BFS: Actual Failure Modes

4.1 What the τ -BFS Detects

The translational invariance check augments every particle position with (τ_r, τ_c) and runs the BFS. If any leaf weight contains τ_r or τ_c , the algorithm is declared translation-non-invariant. This is correct and sound when the algorithm returns to the BFS normalizer (`$dbcNormTauState`) cleanly.

4.2 Internal Mathematica Calls to RandomInteger Cause Spurious Errors

The BFS engine uses `Block[{RandomInteger = ...}]` to intercept all calls to `RandomInteger`. This global interception catches *all* evaluations of `RandomInteger` in the dynamic scope, including internal calls made by Mathematica's own kernel during symbolic evaluation.

In `quadratic_field.wl`, the algorithm normalizes positions internally:

```
newPosNorm = {Mod[newPos[[1]]-1, n]+1, Mod[newPos[[2]]-1, n]+1};
```

When the τ -augmented position $p_i + \tau$ flows through this line, the argument of `Mod` becomes $r + \tau_r + \Delta r - 1$, a symbolic expression. Mathematica’s `PiecewiseExpand` (called by `acceptTestI` on the symbolic ΔE) internally invokes `RandomInteger[{-16, 16}, n]` to sample from a distribution over integer residue classes during its symbolic simplification of `Mod`. This internal call is intercepted by the BFS’s `Block`, does not match any handled form, and throws:

```
$dbc$cantHandle["RandomInteger[{{-16, 16}, 4}]"]
```

The summary line correctly reports “Translational: FAIL (recomputed without)”, and the subsequent DB check is correct. However, the detection mechanism is *not* the intended τ -cancellation check; the BFS merely errored out. The README description “ τ -check detects the non-invariant field” is misleading: τ was never *detected* in any leaf weight. The algorithm fell through to the error path.

Issue 2 (Incorrect τ -detection for quadratic_field). *The τ -BFS for `quadratic_field.wl` reports “FAIL” via a `$dbc$cantHandle` error (internal Mathematica RNG call intercepted), not via the designed mechanism of detecting τ_r or τ_c in leaf weights. The end-state is correct (orbit recomputation without translation), but the diagnostics are wrong and the robustness of the check is lower than claimed.*

This also reveals a design fragility: any algorithm whose evaluation path (when τ -augmented) causes Mathematica to call `RandomInteger` internally will trigger a spurious `cantHandle` error. The τ -BFS cannot distinguish between (a) the user’s algorithm calling `RandomInteger` as part of its logic, and (b) Mathematica’s kernel calling `RandomInteger` during symbolic evaluation.

4.3 The τ -Check from Orbit Representatives Is Sound

The claim that checking τ -freedom from translation-orbit representatives is equivalent to checking from all states is *correct* and the argument is rigorous. For any state $s = T_{dr,dc}(\text{rep})$, running the algorithm from $s + \tau = T_{dr,dc}(\text{rep}) + \tau = T_{dr+\tau_r,dc+\tau_c}(\text{rep})$ is equivalent to running from `rep` with a shifted τ -offset $\tau' = \tau + (dr, dc)$. Since dr, dc are concrete integers that appear only in $\tau' = \tau + \text{integer}$, and τ is free, the τ -BFS output for `rep` subsumes the output for s . If τ cancels for `rep` at any offset, it cancels for `rep` at all offsets.

4.4 The τ -Check Verifies Weights But Not Next-State Consistency

The check tests only `FreeQ[leaf[[3]],  $\tau_r$ ] && \text{FreeQ}[\text{leaf}[[3]], \tau_c]`. It does not directly verify that the next state `leaf[[2]]` is the correct τ -shifted version of the non-augmented next state. For physically sensible algorithms the weight condition is equivalent to translation invariance. For pathological algorithms (position-dependent weights that happen to be τ -free, combined with non-translation-invariant next-state selection), the check could yield a false pass. This edge case is not physical but should be acknowledged.

5 The Ergodicity Check

Issue 3 (Ergodicity BFS starts from `allStates[[1]. ]`, not `$seedState`) In `$dbcCheckErgodicityFromLeaves` the BFS always starts from state index 1:

```
(* BFS from state index 1 (first enumerated state = seed) *)
reachableSet = </1 -> True/;>;
queue = {1};
```

The comment assumes that `allStates[[1]] == $seedState`. This holds for the standard seed-state construction (first N lexicographic positions with sorted types) because `$dbcEnumerateNParticleStates` generates states in the same order. For any user-defined `$seedState` that is not the lexicographically first N -particle state, the ergodicity BFS checks reachability from the wrong starting state.

This is a latent defect. For all provided examples, it is accidentally correct. The fix is to pass `seedState` as a parameter to `$dbcCheckErgodicityFromLeaves` and look up its index in `stateToIdx`.

6 The EBE Exact Check

6.1 Correctness of the Arrangement BFS

The EBE Phase 2 uses a BFS over the face-adjacency graph of the hyperplane arrangement defined by the k Piecewise branch conditions. The correctness relies on:

Claim 1 (Connectivity of hyperplane arrangement chambers). The open chambers (full-dimensional cells) of any arrangement of finitely many hyperplanes in $\mathbb{R}^d$ with rational coefficients form a connected graph under facet-adjacency (two chambers are adjacent iff they share a codimension-1 face).

This claim is *correct*. The standard proof: given any two chambers C_1 and C_2 , connect any interior point of C_1 to any interior point of C_2 by a line segment. The segment crosses a sequence of hyperplanes, and each crossing moves to an adjacent chamber. The resulting sequence of chambers is a path in the adjacency graph from C_1 to C_2 .

Furthermore, for rational-coefficient hyperplanes, every chamber contains a rational point. This guarantees that `FindInstance[... , Rationals]` will find a representative for every feasible region (non-empty chamber). The BFS therefore visits every real chamber.

6.2 Strict Inequalities and Boundary Omission

The Piecewise conditions in the transition weights use strict inequalities ($<$, not $<=$). The EBE arrangement BFS therefore enumerates only the open chambers and does not test the $(d - 1)$ -dimensional boundaries where two or more conditions are simultaneously active.

A coupling configuration on a boundary (e.g., $J_1 = J_2$ exactly) corresponds to a degenerate energy landscape. For a DB violation to exist *only* on such a boundary but not in either adjacent open chamber, the violation would need to be parametric—holding for a measure-zero subset of parameter space.

For Metropolis-type algorithms, this cannot happen: within each open chamber, $T(s \rightarrow t)$ is a specific rational multiple of $e^{-\beta \Delta E}$ and $T(t \rightarrow s)$ is a rational number

or a different Boltzmann factor. By continuity, a violation in an open chamber extends to the boundary of that chamber. Since EBE checks all open chambers, any violation that extends to an open neighborhood will be detected.

The argument breaks for algorithms with *discontinuous* transition rules (e.g., a rule that is applied only when $J_1 = J_2$ exactly), but such algorithms have measure-zero probability in practice. This should be documented explicitly.

6.3 Correctness of \$dbcIsExpZero

The function checks whether a symbolic expression of the form $\sum_i c_i e^{-\beta E_i}$ is identically zero (for symbolic $\beta > 0$, rational c_i and E_i). The method:

1. `$dbcMergeExp`: merge products $e^x e^y \rightarrow e^{x+y}$ using `FixedPoint` with up to 30 iterations.
2. `$dbcSplitTerm`: split each term into (rational coefficient, exponent).
3. `$dbcGroupByExp`: group terms by exponent using `Expand[...]` `=== 0`.
4. Check that the sum of rational coefficients in each exponent group is zero.

This is *correct* for the specific structure arising in Metropolis-type algorithms (rational coupling constants substituted via J^* , β remaining symbolic). After J^* substitution, all energy values are rational, so all exponents are of the form $-\beta \times \text{rational}$.

However, the function returns `$dbcFS` (fallback sentinel) when a term has multiple non-merged `Exp` factors or a transcendental coefficient. This sentinel is treated as a non-True result:

```
If[res != True,
  violations[pair] = <| ... |>]
```

Any such `$dbcFS` result adds a spurious entry to the violations list, constituting a *false positive*. The user would be informed of a DB violation that is in fact an analysis failure, with no diagnostic distinguishing the two cases.

7 Minor Issues

7.1 Time-Limited BFS Returns Partial Leaves Without Error

When the per-state time limit `tlim` is exceeded in `$dbcBuildStateLeaves`, the function returns whatever leaves have been collected so far with only a warning print:

```
If[timedOut, Print["␣␣WARNING:␣time␣limit␣reached␣for␣state␣",
  state]];
leaves]
```

This is a correctness hazard: the subsequent DB check silently operates on an incomplete transition matrix. A timed-out BFS should either raise a hard error (like the `maxDepth` case) or the final verdict should be marked as `INCONCLUSIVE` rather than `PASS` or `FAIL`.

7.2 Coupling Symmetry Rule Persists Across Runs in a Live Kernel

The call `$dbcAddCouplingSymmetry[symCouplings]` installs a global rewrite rule for the coupling head. If the same kernel processes multiple algorithm files without restarting, and the second file has a different coupling structure, residual rules from the first run may interfere. The `check.wls` script calls `ClearAll` on the coupling head before reinstalling, which mitigates this in the script context. Users who load `dbc_core.wl` interactively should be aware of this issue.

7.3 The seqBernoulli Weight Calculation Is Correct But Fragile

The sequential Bernoulli decomposition of `RandomChoice[weights -> elements]` is mathematically correct. However, when `Total[ws] = 0` (all weights are zero, which can occur for Piecewise weight expressions that are zero in the current coupling region), division by zero occurs in the computation `p = ws[[i]] / remainW`. This would produce `ComplexInfinity` or `Indeterminate`, potentially corrupting subsequent computations. No guard is present.

7.4 The RandomVariate[NormalDistribution] Approximation

The BFS supports `RandomVariate[NormalDistribution[ $\mu$ ,  $\sigma$ ]]` by discretizing the distribution onto integer values near μ , using `seqBernoulli` with CDF-based weights. The approximation truncates at `nMax =  $\lfloor n_{\text{Grid}}/2 \rfloor$` above and below μ . For a 3×3 grid, this is ± 1 , which severely underestimates the tails of any distribution with $\sigma \geq 1$. The approximation error in the weights is unbounded relative to the exact normal distribution. This discretization is not documented and its accuracy is not checked.

8 Documentation Inconsistencies

8.1 Help Text vs. Code: ebeMaxK Default

In `check.wls` (line 19), the usage comment states:

```
-ebeMaxK N           Max branch conditions for EBE; falls back to SZ
      above (default 12)
```

However, the code at line 48 defaults to 50:

```
ebeMaxK = ToExpression[parseOpt["-ebeMaxK", "50"]];
```

The README correctly documents the default as 50. The comment is wrong and should be corrected to avoid user confusion.

8.2 Description of τ -Failure for quadratic_field.wl

The README states: “Translational: FAIL (recomputed without) $\leftarrow$ field breaks τ -invariance; expected.” The actual mechanism is that the τ -BFS errors out with a `canHandle` exception (Mathematica’s internal `RandomInteger` call is intercepted), not that τ is detected in leaf weights. The result is the same, but the description implies a more principled detection than actually occurs.

8.3 Claim About False-Positive Probability

The README states: “A false positive ...would require the T matrix symmetry to hold at 10 independent random coupling points by coincidence—negligibly unlikely.” For genuinely asymmetric algorithms, this is correct (the zero set of a nonzero polynomial has measure zero). However, for algorithms that are D4-symmetric up to a small perturbation ϵ , the violation magnitude may be $O(\epsilon)$, which could be below the numerical tolerance 10^{-6} for sufficiently small ϵ . The claim should be qualified: it holds for algorithms whose D4 violation is not numerically small relative to the tolerance.

9 Fixes Applied and Remaining Issues

Following the initial audit, the three highest-impact issues were addressed in the codebase. This section records what was changed, verified, and what remains open.

9.1 Fix 1: RandomChoice weight correction (critical)

Root cause. `RunWithBitsAT` reads $k = \lceil \log_2 n \rceil$ bits to choose uniformly from an n -element list. Each `readBit[]` call multiplies the path weight by $1/2$, giving total weight $(1/2)^k$ per valid leaf. For n not a power of two this is $1/2^k < 1/n$, so all leaf weights are underestimated by a factor $n/2^k$.

Fix. After a successful index read (`idx < n`), the path weight is multiplied by the correction factor $2^k/n$:

```
(* before *)
If[idx>=n, Throw[$dbc$outOfRange,...], list[[1+idx]]]
(* after *)
If[idx>=n, Throw[$dbc$outOfRange,...], weight*=2^k/n; list[[1+idx]]]
```

The same correction was applied to both `RandomInteger[{lo,hi}]` and `RandomInteger[{0,n-1}]`, which share the same rejection-sampling logic. For n a power of two, $2^k = n$ and the correction is 1 (no change).

Verification with purpose-built failing examples. Two new example files were added to the repository to concretely demonstrate the bug and confirm the fix:

1. `broken_variable_pool.wl` (4-connectivity hop, $k = 2$ bracket). Two particles on a 3×3 torus. The chosen particle hops to a randomly selected *unoccupied* 4-neighbour. When the particles are adjacent, the pool has 3 elements; when non-adjacent, 4 elements. Both $n = 3$ and $n = 4$ use $k = 2$ bits.

True probabilities: $T(\text{adj} \rightarrow \text{nonadj}) = \frac{1}{2} \cdot \frac{1}{3}$, $T(\text{nonadj} \rightarrow \text{adj}) = \frac{1}{2} \cdot \frac{1}{4}$. With energy zero (uniform π), detailed balance requires these to be equal, which they are not — a genuine DB violation.

Before fix: BFS assigns $\frac{1}{2} \cdot \frac{1}{4}$ to both transitions $\Rightarrow$ false **PASS**. After fix: BFS assigns $\frac{1}{2} \cdot \frac{1}{3}$ vs $\frac{1}{2} \cdot \frac{1}{4} \Rightarrow$ correct **FAIL**, with $T(s \rightarrow t) = 1/6$ and $T(t \rightarrow s) = 1/8$ printed.

2. `broken_8way_hop.wl` (8-connectivity hop, $k = 3$ bracket). Two particles on a 4×4 torus (`nGrid=4` is required; on the 3×3 torus every pair of sites is within 8-distance

1, making the pool uniformly 7 and the algorithm accidentally correct). When the particles are 8-adjacent, pool = 7; otherwise pool = 8. Both use $k = 3$ bits.

Before fix: BFS assigns $\frac{1}{2} \cdot \frac{1}{8}$ to both $\Rightarrow$ false **PASS**. After fix: $\frac{1}{2} \cdot \frac{1}{7}$ vs $\frac{1}{2} \cdot \frac{1}{8} \Rightarrow$ correct **FAIL**, with $T(s \rightarrow t) = 1/14$ and $T(t \rightarrow s) = 1/16$.

Regression tests. All five pre-existing examples (`single_metropolis.wl`, `kawasaki.wl`, `vmmc_2d.wl`, `quadratic_field.wl`) produce identical verdicts before and after the fix. In `kawasaki.wl`, `RandomChoice` selects from 3 distinct-type pairs on *both* the forward and reverse transition, so the correction factor 4/3 appears symmetrically and cancels — confirming that the previous accidental correctness was fragile rather than principled.

9.2 Fix 2: Ergodicity BFS starts from correct seed state

Root cause. The reachability BFS in `$dbcCheckErgodicityFromLeaves` was hardcoded to start from state index 1 (`allStates[[1]]`), not from `$seedState`. For all provided examples `$seedState` happened to coincide with `allStates[[1]]` (the lexicographically first state), so the bug was latent.

Fix. The function signature was extended to accept the seed state as a required parameter:

```
(* before *)
$dbcCheckErgodicityFromLeaves[... , allStates_ , nGrid_] :=
  Module[... ,
    reachableSet = <|1 -> True|>; queue = {1};

(* after *)
$dbcCheckErgodicityFromLeaves[... , allStates_ , nGrid_ , seedState_]
:=
  Module[... ,
    seedIdx = Lookup[stateToIdx , Key[seedState] , 1];
    reachableSet = <|seedIdx -> True|>; queue = {seedIdx};
```

The call site in `check.wls` was updated to pass `seedState`. A fallback to index 1 is retained via `Lookup[... , 1]` in case the seed is not found in the state index (which should not occur for well-formed input).

9.3 Fix 3: ebeMaxK documentation error

The `-ebeMaxK` help text in `check.wls` stated “(default 12)” while the actual default was 50. The help text was corrected to “(default 50)”.

9.4 All four open issues now resolved

All four issues listed in the original audit have been addressed in a further revision.

1. **D4 check made algebraically exact (Issue 1).** A new function `$dbcEBECheckD4` performs the D4 check via the same EBE Phase 1–3 machinery as the DB check. Within each feasible parameter region it checks

$$\text{\$dbcIsExpZero}[T(s \rightarrow t) - T(R \cdot s \rightarrow R \cdot t)] = \text{True}$$

for all communicating pairs (s, t) . Only two D4 generators are checked (rot90CW and LR-reflection, rotIdx = 1 and 4); by the group-theoretic argument, invariance under both generators implies invariance under the full D4 group. If $k > \text{ebeMaxK}$ the function falls back to the previous probabilistic check.

The EBE chambers computed during the D4 check are passed to the DB check, which reuses them to skip its own Phase 2 enumeration. For `single_metropolis.wl` ($k = 18$, 132 regions) this reduces total EBE time from $76 + 38 = 114$ s to $39 + 29 = 68$ s. For `vmmc_2d.wl` ($k = 12$, 512 regions) the D4 check takes ≈ 238 s (four-fold improvement over the original 7-rotation implementation).

2. **τ -BFS cantHandle for in-body Mod improved (Issue 2).** The `RandomInteger` Block override now handles the vectorised form `RandomInteger[{lo,hi}, count]` (and `RandomInteger[n, count]`) by generating each of the `count` integers independently via the rejection-sampling bit stream. If the total bit cost $\text{count} \times \lceil \log_2 n \rceil > 20$, the call falls back to `cantHandle` with an informative message.

This fixes algorithms that genuinely use multi-integer generation. For `quadratic_field.wl`, the internal Mathematica call `RandomInteger[{-16,16},4]` has bit cost $24 > 20$ and is therefore caught by the new threshold, producing a clearer error message:

```
RandomInteger[{{-16, 16}, 4}]: 24 bits needed -- use count x RandomInteger[{lo,hi},
instead
```

The underlying cause (Mathematica's internal evaluation of symbolic `Mod` intercepted by the Block override) remains a known limitation; the τ -BFS result for `quadratic_field.wl` is still correct (via the `cantHandle` path).

3. **Timed-out BFS now aborts with error (Issue 3).** `$dbcBuildStateLeaves` now returns a `$dbc$cantHandle` sentinel when the per-state time limit is exceeded, rather than printing a warning and returning partial leaves. This causes the run to abort at `step 5` (or at the τ -BFS step) with a clear message instructing the user to increase `-timeLimit`.
4. **seqBernoulli zero-weight guard added (Issue 4).** A guard `If[TrueQ[remainW == 0], Throw[$dbc$cantHandle[...]]]` is now placed at the start of `seqBernoulli`. If all weights are numerically zero the function throws a `cantHandle` error rather than dividing by zero.

9.5 Summary of status after second revision

Issue	Status	Notes
Weight bug: RandomChoice non-power-of-2	Fixed (1st revision)	
D4 check numerical	Fixed (2nd revision)	Algebraic via EBE
τ -BFS cantHandle (internal call)	Partially fixed	Better error msg
Ergodicity BFS wrong seed	Fixed (1st revision)	
Timed-out BFS returns partial leaves	Fixed (2nd revision)	Now aborts
seqBernoulli zero-weight	Fixed (2nd revision)	
ebeMaxK help text wrong	Fixed (1st revision)	

The sole remaining structural limitation is that `quadratic_field.wl`-style algorithms (with in-body `Mod` on symbolic positions) cannot be verified for τ -invariance via the algebraic BFS path; the checker falls back to the correct result via the `cantHandle` path, but with a less informative error chain than ideal. Resolving this would require preventing Mathematica’s Block-scope override from intercepting internal kernel evaluation calls, which is not feasible in standard Mathematica.

10 Summary Table

Issue	Severity	Fixed?	Affects examples?
Weight bug: non-power-of-2 <code>RandomChoice</code>	Critical	R1	No (cancels)
D4 check is numerical/probabilistic	Significant	R2	No
τ -BFS errors out (not τ -detection)	Moderate	Partial (R2)	Yes
Ergodicity BFS starts from wrong state	Moderate	R1	No (coincides)
<code>\$dbcFS</code> treated as violation (false positive)	Moderate	–	No
Timed-out BFS returns incomplete leaves	Moderate	R2	No
<code>seqBernoulli</code> : no division-by-zero guard	Minor	R2	No
<code>NormalDistribution</code> truncation not validated	Minor	–	No
<code>ebeMaxK</code> help text says 12 (should be 50)	Documentation	R1	–
τ -failure description in README	Documentation	R1 (rewritten)	–

Table 1: Summary of findings. R1 = fixed in first revision; R2 = fixed in second revision; – = not yet fixed. See Section 9.

11 Conclusions

Translational invariance check. The τ -BFS approach is mathematically sound. Checking from orbit representatives is provably equivalent to checking from all states. However, the detection mechanism can silently switch from “ τ found in weights” to “`cantHandle` error” (for algorithms that trigger internal Mathematica random-number generation), giving correct output via an incorrect path.

EBE detailed balance check. The EBE machinery is correct for Metropolis-type algorithms with rational coupling constants. The arrangement BFS correctly enumerates all feasible parameter regions via the hyperplane connectivity theorem. The `$dbcIsExpZero` check correctly handles all expressions arising from standard Metropolis acceptance. The check is *exact* (not probabilistic) for $k \leq \text{ebeMaxK}$.

Critical weight bug. The transition weights assigned by the BFS for `RandomChoice[list]` with non-power-of-2 n are systematically off by a factor $n/2^k$. For the provided examples this factor cancels between forward and reverse transitions, leaving the DB verdict correct. For algorithms with varying proposal-pool sizes (asymmetric Metropolis-Hastings), this bug can produce both false positives and false negatives. It must be fixed before the checker can be applied with confidence to general Metropolis-Hastings algorithms.

D4 check. The numerical D4 check cannot be replaced by a τ -style algebraic trick due to the discrete, non-additive nature of D4. However, an exact certification is achievable by extending the existing EBE machinery: after enumerating all parameter regions, check

$T(s \rightarrow t) = T(R \cdot s \rightarrow R \cdot t)$ as exact rational equalities within each region. This would make the D4 check as rigorous as the DB check itself, at comparable computational cost.